RISC-V

리스크 파이브[1]

개발	UC 버클리 RISC-V International
발표	2010년
분류	명령어 집합(RISC)
비트	32, 64, 128비트
링크	

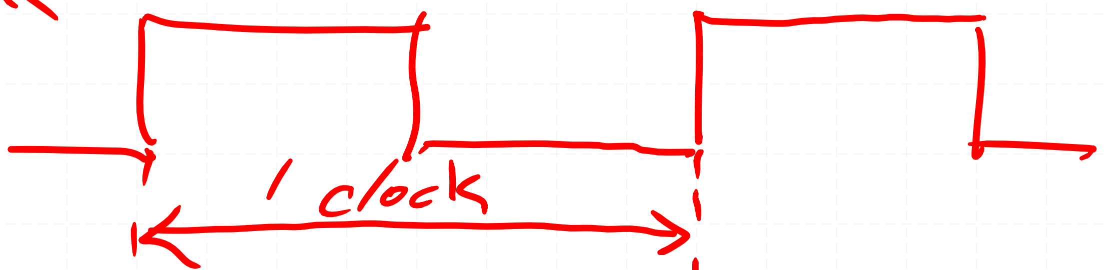
RISC-V: The Free and Open RISC Instruction Set Architecture

2010년부터 미국의 UC 버클리에서 개발중인 무료 오픈 소스 RISC 명령어셋 아키텍처다.

Base	Version	Frozen?
RV32I	2.0	Y
RV32E	1.9	N
RV64I	2.0	Y
RV128I	1.7	N
Extension	**Version**	**Frozen?**
M	2.0	Y
A	2.0	Y
F	2.0	Y
D	2.0	Y
Q	2.0	Y
L	0.0	N
C	2.0	Y
B	0.0	N
J	0.0	N
T	0.0	N
P	0.1	N
V	0.2	N
N	1.1	N

1. Single-Cycle Architecture

clock



모든 명령어가 1/clock에 동작

장점: 구조가 매우 simple하다.

단점: 느리다.

2. Multi-Cycle Architecture

- 명령어 type별로 동작clock 수가 다르다

장점: Single-Cycle보다 조금 빠르다.

단점: Single-Cycle보다 구조가 조금 복잡하다.

3. Pipe-Line Architecture (X)

장점: Single-Cycle보다 많이 빠르다.

단점: 〃 구조가 많이 복잡하다

RV32I

32bit

XLEN-1	0
x0 / zero ø	
x1	
x2	
x3	
x4	
x5	
x6	
x7	
x8	
x9	
x10	
x11	
x12	
x13	
x14	
x15	
x16	
x17	
x18	
x19	
x20	
x21	
x22	
x23	
x24	
x25	
x26	
x27	
x28	
x29	
x30	
x31	

XLEN

XLEN-1	0
pc	

XLEN

Program Counter

==

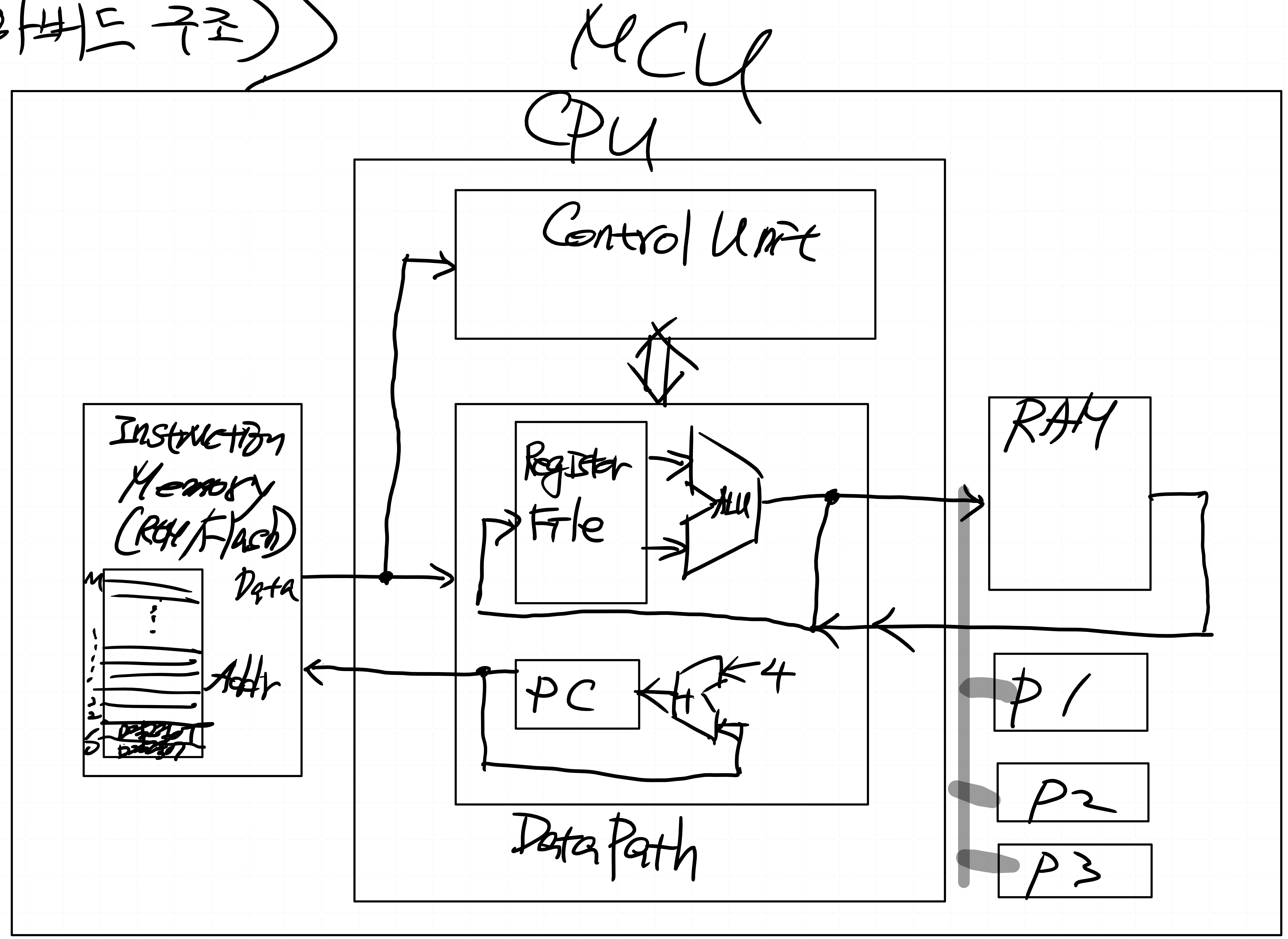
32bit

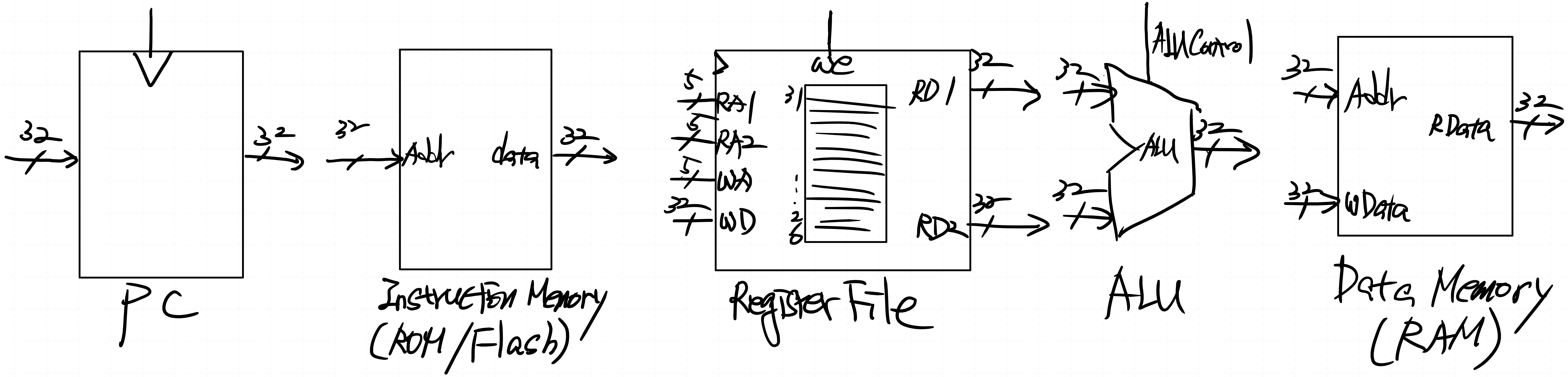
31 — 25	24 — 20	19 — 15	14 — 12	11 — 7	6 — 0	
funct7	rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]		rs1	funct3	rd	opcode	I-type
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[31:12]				rd	opcode	U-type

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5	t0	Temporary/alternate link register	Caller
x6–7	t1–2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10–11	a0–1	Function arguments/return values	Caller
x12–17	a2–7	Function arguments	Caller
x18–27	s2–11	Saved registers	Callee
x28–31	t3–6	Temporaries	Caller
f0–7	ft0–7	FP temporaries	Caller
f8–9	fs0–1	FP saved registers	Callee
f10–11	fa0–1	FP arguments/return values	Caller
f12–17	fa2–7	FP arguments	Caller
f18–27	fs2–11	FP saved registers	Callee
f28–31	ft8–11	FP temporaries	Caller

< CPU 기본 모듈 (하버드 구조) >

- Register File
- ALU
- ROM/Flash
  (Instruction Memory)
- RAM
  (Data Memory)
- PC
  (Program Counter)





ALU Control	Fuction
00	ADD
01	SUB
10	AND
11	OR

(임시값)

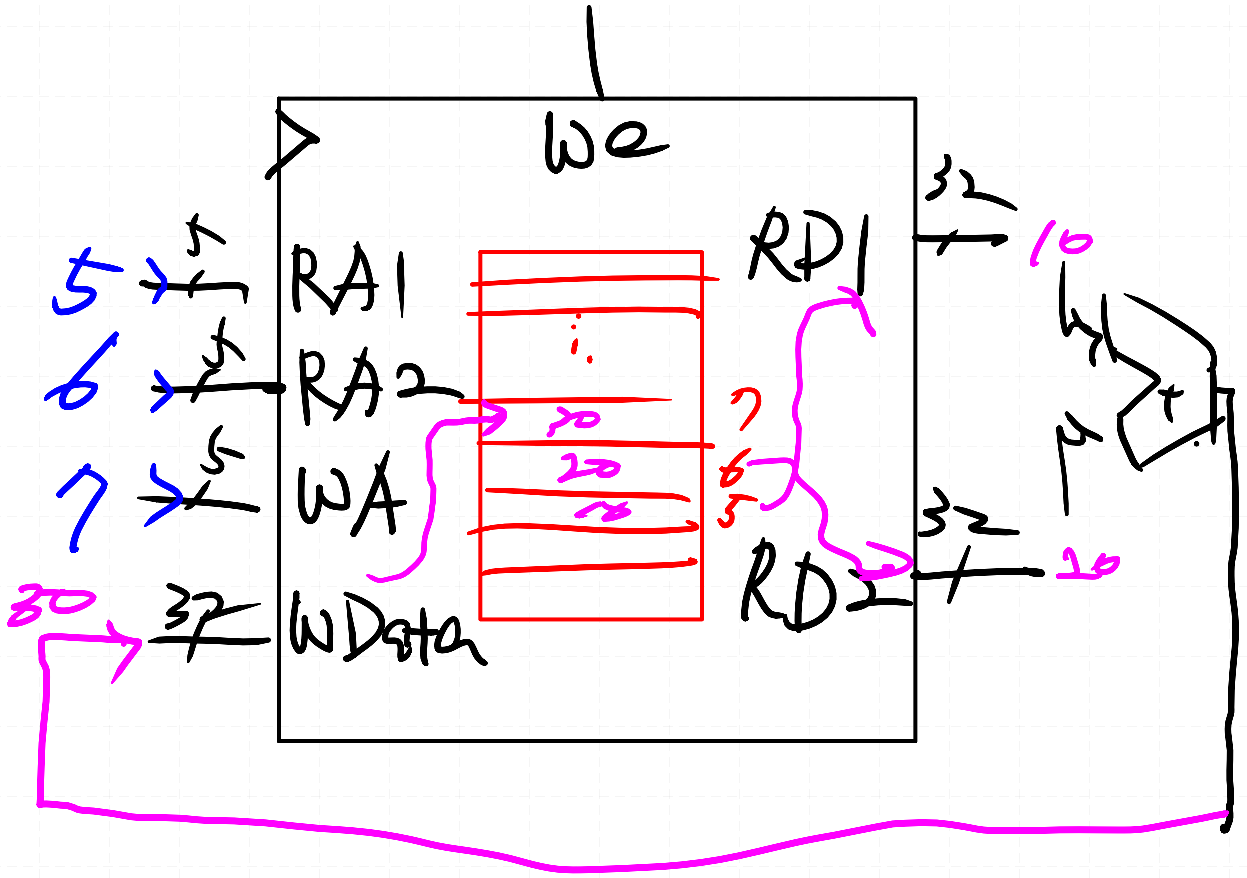
Register File

TYPE	MNEMONIC	NAME	Descript
R-TYPE	ADD	ADD	rd = rs1 + rs2
	SUB	SUB	rd = rs1 - rs2
	SLL	Shift Left Logical	rd = rs1 << rs2
	SRL	Shift Right Logical	rd = rs1 >> rs2
	SRA	Shift Right Arith*	rd = rs1 >>> rs2
	SLT	Set Less Than	rd = (rs1 < rs2) ? 1:0
	SLTU	Set Less Than (U)	rd = (rs1 < rs2) ? 1:0
	XOR	XOR	rd = rs1 ^ rs2
	OR	OR	rd = rs1 | rs2
	AND	AND	rd = rs1 & rs2

ADD $\underbrace{rd}_{\text{WAddr}} = \underbrace{rs1}_{\text{RA1}} + \underbrace{rs2}_{\text{RA2}}$

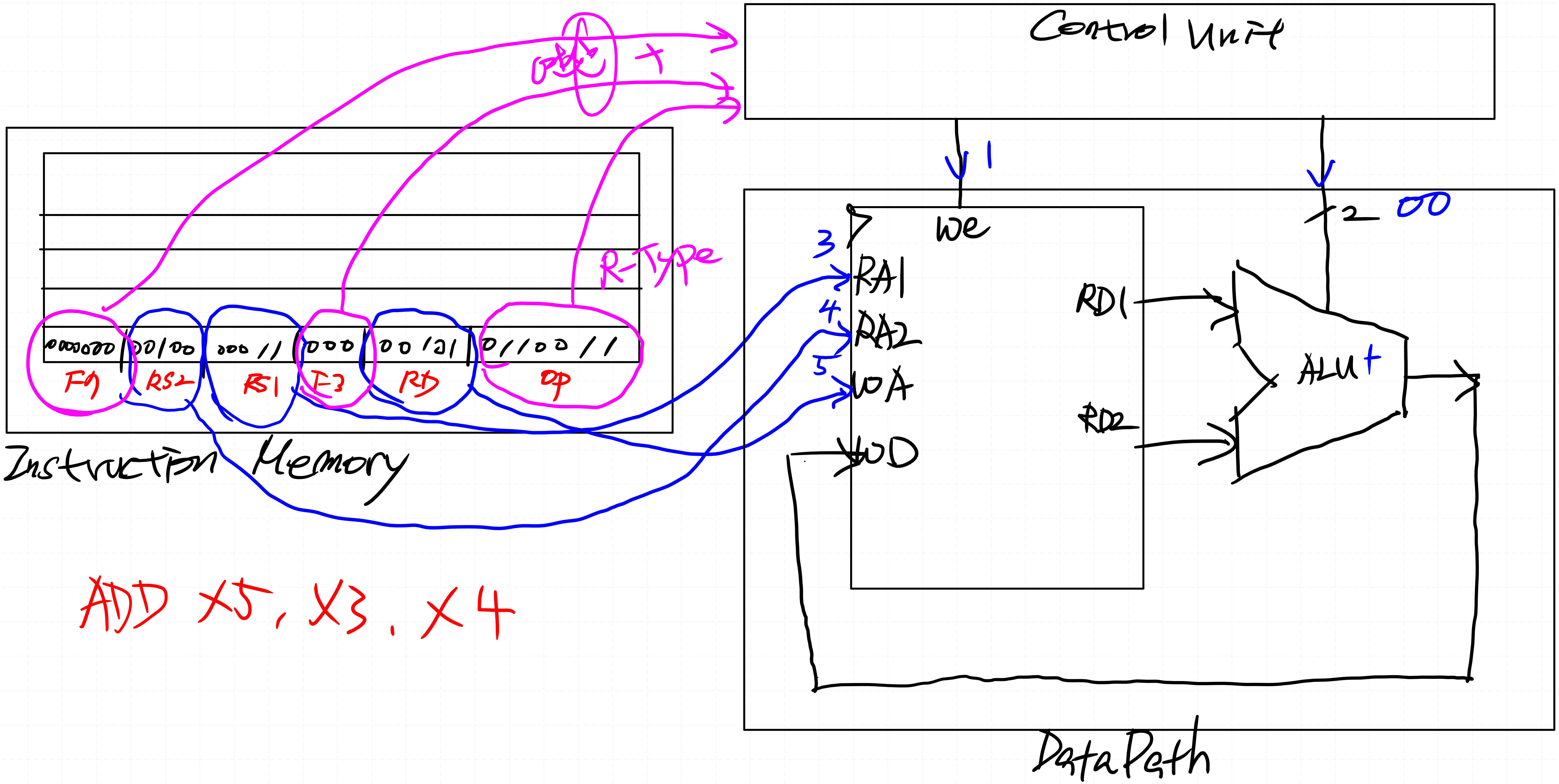
ADD $\underbrace{x7}_{rd}$, $\underbrace{\overset{10}{x5}}_{rs1}$, $\underbrace{\overset{20}{x6}}_{rs2}$

$x7 = \underbrace{x5}_{\text{Data}} + \underbrace{x6}_{\text{Data}}$



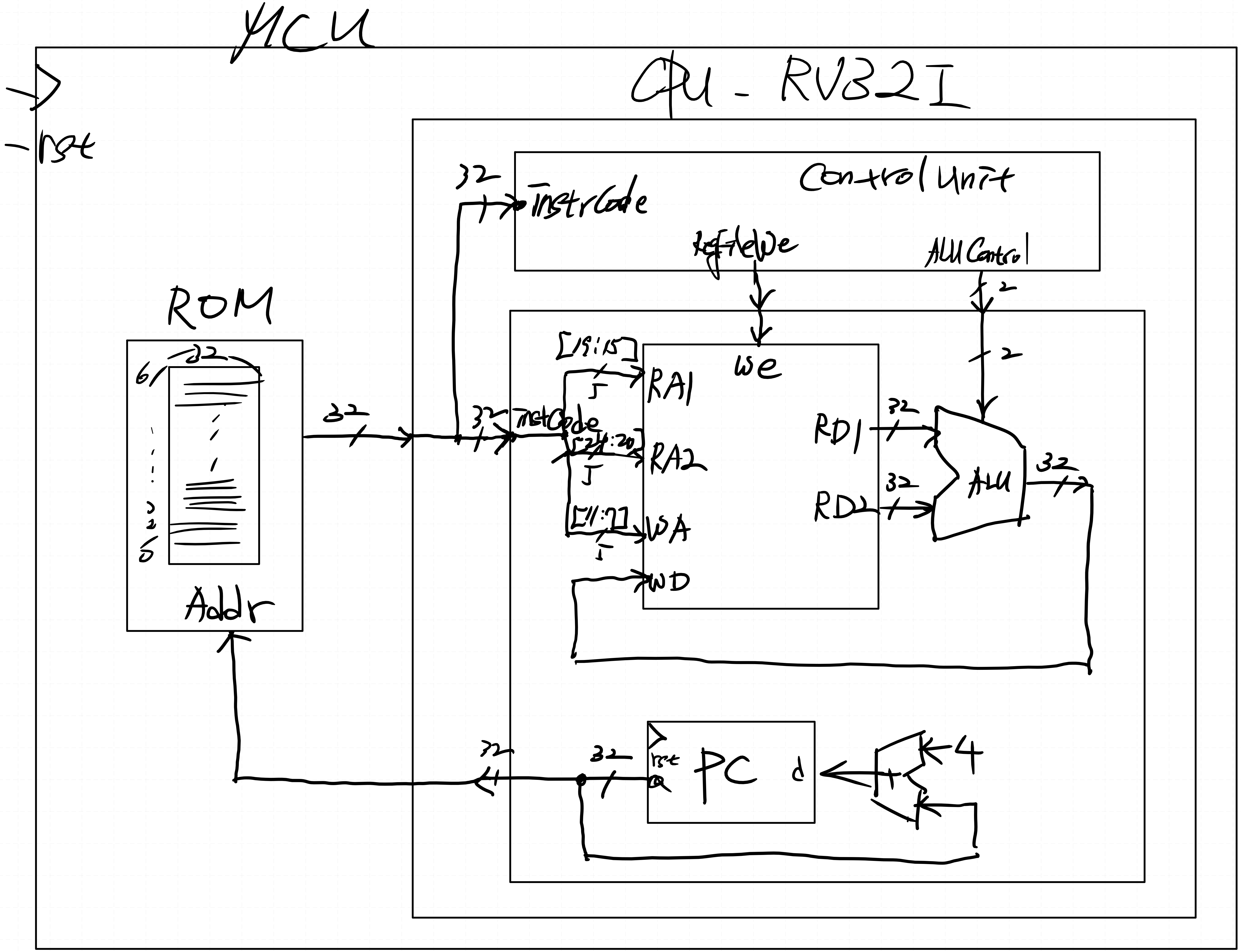
RS2 RS1 RD OP

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	TYPE	MNEMONIC	NAME
			Funct7 (7)						Regitster Source 2 (5)					Regitster Source 1 (5)				Funct3 (3)				Register Destination (5)						Opcode (7)						
0	0	0	0	0	0	0			rs2					rs1			0	0	0			rd			0	1	1	0	0	1	1	R-TYPE	ADD	ADD
0	1	0	0	0	0	0			rs2					rs1			0	0	0			rd			0	1	1	0	0	1	1		SUB	SUB
0	0	0	0	0	0	0			rs2					rs1			0	0	1			rd			0	1	1	0	0	1	1		SLL	Shift Left Logical
0	0	0	0	0	0	0			rs2					rs1			1	0	1			rd			0	1	1	0	0	1	1		SRL	Shift Right Logical
0	1	0	0	0	0	0			rs2					rs1			1	0	1			rd			0	1	1	0	0	1	1		SRA	Shift Right Arith*
0	0	0	0	0	0	0			rs2					rs1			0	1	0			rd			0	1	1	0	0	1	1		SLT	Set Less Than
0	0	0	0	0	0	0			rs2					rs1			0	1	1			rd			0	1	1	0	0	1	1		SLTU	Set Less Than (U)
0	0	0	0	0	0	0			rs2					rs1			1	0	0			rd			0	1	1	0	0	1	1		XOR	XOR
0	0	0	0	0	0	0			rs2					rs1			1	1	0			rd			0	1	1	0	0	1	1		OR	OR
0	0	0	0	0	0	0			rs2					rs1			1	1	1			rd			0	1	1	0	0	1	1		AND	AND

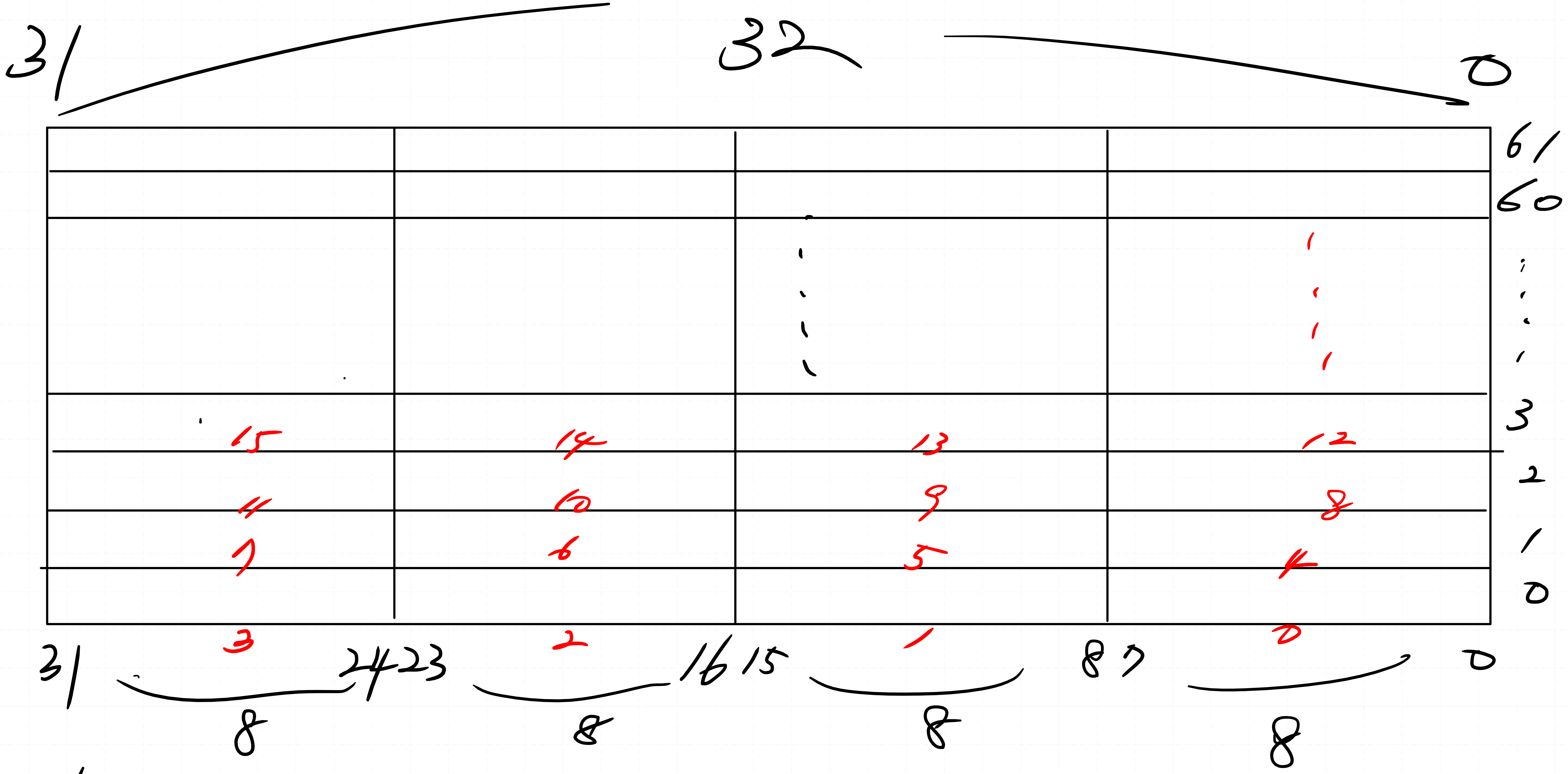


Func7 RS2 RS1 Func3 RD OP

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	TYPE	MNEMONIC	
Funct7 (7)							Regitster Source 2 (5)					Regitster Source 1 (5)					Funct3 (3)			Register Destination (5)					Opcode (7)									
0	0	0	0	0	0	0	rs2 4					rs1 3					0	0	0	rd 5					0	1	1	0	0	1	1		ADD	ADD
0	1	0	0	0	0	0	rs2 9					rs1 8					0	0	0	rd 7					0	1	1	0	0	1	1		SUB	SUB



MCU
rst
CPU - RV32I
Control Unit
InstrCode
RegFileWe
ALU Control
ROM
Addr
we
RA1
RA2
WA
WD
RD1
RD2
ALU
PC
rst
q
d
4
[19:15]
[24:20]
[11:7]
32
2



4b 1000 : 8 2

4b 0100 : 4 1

4b 0000 : 0 0

5b 10000 : 16 4

4b 1100 : 12 3


clk
reset
ControlUnit
instrCode[31:0]
regFileWe
aluControl[1:0]
opcode[6:0]
operator[3:0]
DataPath
instrCode[31:0]
instrMemAddr[31:0]
regFileWe
aluControl[1:0]
aluResult[31:0]
RFData1[31:0]
RFData2[31:0]
PCSrcData[31:0]
PCOutData[31:0]
RegFile
mem[0:31][31:0]
1
0
XXXXXXXX
0
X
XX
X
XXXXXXXX
00000018
0
X
XXXXXXXX
XXXXXXXX
XXXXXXXX
0000001c
00000018
0000000a,000
00110233 add
401102b3 sub
0001f333 and
0001e3b3 or
add x4,x2,x1
sub x5,x2,x1
and x6,x3,x0
or x7,x3,x0
0 add
1
2
3
33 R-Type
0 add
8
7
6
00110233
401102b3
0001f333
0001e3b3
00000000 0
00000004
00000008
0000000c
0 add
1
2
3
00000017 12+11
00000001
00000000
0000000d
12 0000000c
0000000d
11 0000000b
00000000
00000004 4
00000008
0000000c
00000010
00000000
0
4 00000004
00000008
0000000c 12
0000000a,0000000b,0000000c,0000000d,00000017,0000000f,...
0000000a,0000000b,0000000c,00,..
0000000a,0000000b,0000000c,00,..

R-Type 모두 구현하시오.

코드, 타이밍 다이어그램 시뮬레이션결과

Classroom에 업로드 하시오.

쉬는날 꼭! 복습할것!!

칠판 | 칠판

강사자리

1	2	3	4	5	6	7	8	9	10	11	12
13	14	15	16	17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32	33	34	35	36
37	38	39	40	41	42	43	44				

☆ 엄찬하 #1
✓ 장찬원 #2
✓ 권혁진 #3
✓ 최규리 #4
✓ 임도엽 #5
✓ 장 환 #6
✓ 권희식 #7
✓ 박승헌 #8
✓ 신상학 #9
✓ 고종완 #10
✓ 서윤철 #11
✓ 정은지 #12

✓ 김울중 #13
✓ 김종현 #14
✓ 김민규 #15
✓ 이은성 #16
✓ 문우진 #17
✓ 박지훈 #18
✓ 안재용 #19
✓ 김한벗 #20
✓ 김진수 #21
✓ 임승빈 #22
✓ 이현수 #23
✓ 송유경 #24

✓ 윤의빈 #25
✓ 정현태 #26
✓ 김병현 #27
✓ 정민교 #28
✓ 최지우 #29
✓ 최윤석 #30
✓ 최현우 #31
✓ 김용호 #32
✓ 강석현 #33
✓ 박승범 #34
✓ 양현준 #35
✓ 장희재 #36

✓ 윤종민 #37
✓ 배상원 #38
✓ 변준섭 #39
✓ 임재홍 #40
✓ 김지석 #41
✓ 오정일 #42
✓ 정지원 #43
✓ 조성현 #44